CSED233 · 데이터구조 · 포스텍

정리본 10. AVL Tree

Balance Factor · 4가지 회전 유형 · Insert/Delete 재균형.

1. 왜 AVL이 필요한가

정리본 9에서 봤듯, 편향 BST는 높이가 최대 n−1까지 커져 모든 연산이 $O(n)$이 된다. 높이를 강제로 제한하면 모든 연산을 $O(\log n)$으로 보장할 수 있다 — 그게 자동 균형 트리다.

AVL Tree

다음 두 조건을 모두 만족하는 이진 트리:

1. **BST 성질**: 왼쪽 < 부모 < 오른쪽.
2. **Height-Balanced 성질**: 모든 노드에서 좌우 서브트리 높이 차가 1 이하.

높이와 Balance Factor

$$h(x) = 1 + \max(h(\text{left}), h(\text{right})), \quad h(\text{null}) = -1$$

$$bf(x) = h(\text{left}) - h(\text{right})$$

AVL 조건: 모든 노드에서 $-1 \le bf(x) \le 1$. |bf| ≥ 2이면 불균형 → 회전 필요.

- bf = −1: 오른쪽이 1 높음 (OK) · bf = 0: 완전 균형 · bf = +1: 왼쪽이 1 높음 (OK)

이 조건 덕분에 n개 노드의 AVL 트리는

$$h \le 1.44 \log_2 n$$

를 만족해 편향이 불가능하다.

왜 로그 높이가 보장되는지 보려면, 높이 h인 AVL 트리가 가질 수 있는 최소 노드 수 N(h)를 세면 된다. 가장 기울어진 AVL 트리에서도 N(0)=1, N(1)=2, N(h)=N(h-1)+N(h-2)+1을 만족하므로 노드 수는 피보나치식으로 증가한다. 따라서 n개의 노드가 있으면 높이는 대략 h ≤ 1.44 log₂(n+2) 수준으로 제한된다.

2. 불균형의 4가지 유형과 회전

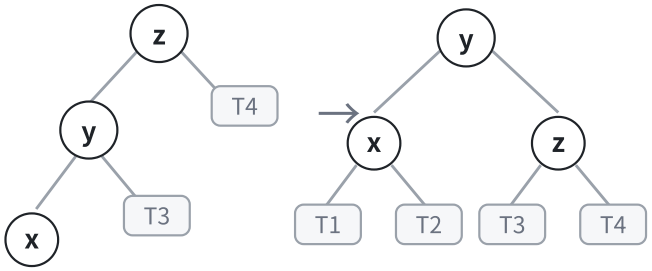
노드 x에서 |bf(x)| = 2가 발견되면, **무거운 쪽 자식의 bf 부호**로 유형을 판정한다.

유형	판정	의미	해결
LL	bf(x) = +2, bf(왼쪽 자식) ≥ 0 (삭제 재균형에서는 =0도 LL)	왼쪽-왼쪽으로 무거움	**우회전 1회**
RR	bf(x) = −2, bf(오른쪽 자식) ≤ 0 (삭제 재균형에서는 =0도 RR)	오른쪽-오른쪽으로 무거움	**좌회전 1회**
LR	bf(x) = +2, bf(왼쪽 자식) < 0	왼쪽 자식의 오른쪽이 무거움	**자식 좌회전 → 부모 우회전**
RL	bf(x) = −2, bf(오른쪽 자식) > 0	오른쪽 자식의 왼쪽이 무거움	**자식 우회전 → 부모 좌회전**

삭제 재균형에서는 무거운 쪽 자식의 bf가 0인 경우도 LL·RR로 처리한다. 이 경우 회전 후 해당 서브트리의 높이는 변하지 않으므로 불균형 전파는 거기서 멈춘다.

2.1 Single Rotation 메커니즘

Right Rotation (우회전)



1. y의 오른쪽 자식(T3)이 z의 왼쪽 자식이 됨
2. y가 z의 자리를 대체
3. z가 y의 오른쪽 자식이 됨

회전 후에도 BST 성질(inorder 순서)은 유지됨

Left Rotation은 이 그림의 좌우 대칭

```
rotateRight(z):
 y = z.left       // 나중에 접근 못 하니 먼저 저장
 z.left = y.right
 y.right = z
 update_height(z)  // 아래쪽(자식이 된 z)부터
```

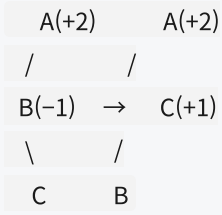
```
update_height(y)
return y        // 새 서브트리 루트
```

또한 rotateRight의 반환값 y는 호출자가 부모의 자식 포인터(또는 루트)에 다시 연결해야 한다.

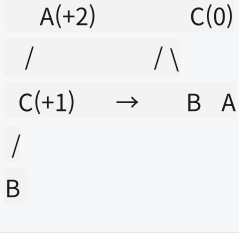
2.2 Double Rotation — LR 타입

예제 · LR: 왼쪽은 무거운데(bf=+2) 왼쪽 자식은 오른쪽이 무거움(bf=−1)

① **자식 좌회전** — B를 기준으로 좌회전하면 A–C–B가 일직선 LL 형태가 된다.



② **부모 우회전** — A를 기준으로 우회전하면 C가 새 루트가 되고 균형 복구.



RL 타입은 정확히 대칭: "자식 우회전 → 부모 좌회전".

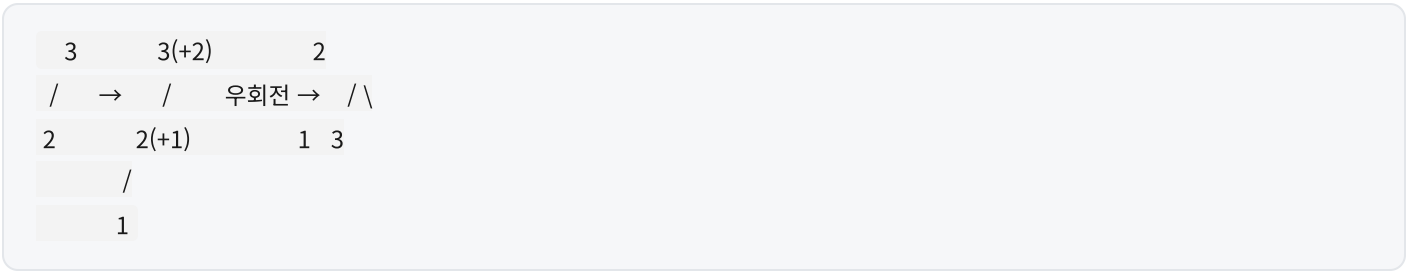
Double rotation의 첫 회전은 **안쪽으로 꺾인 다리를 펴는** 작업이다. 펴진 뒤(LL/RR)에는 single rotation 한 번으로 끝.

3. Insert — 삽입 후 재균형은 최대 1회

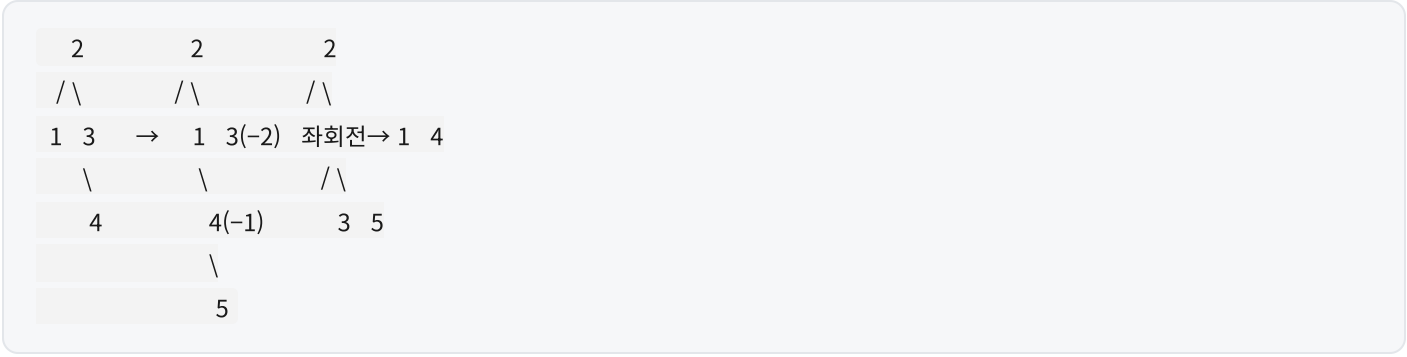
① **표준 BST Insert**: 삽입 위치를 찾아 리프에 노드를 추가한다.

② **상향 조정**: 삽입 노드에서 루트까지 올라가며 높이와 bf를 갱신하고, |bf| = 2인 첫 노드에서 유형에 맞는 재균형을 한 번 수행하면 삽입 전의 높이가 복원되어 전파가 멈춘다. 이때 single rotation은 회전 1회, double rotation(LR·RL)은 회전 2회로 센다.

예제 1 · Insert 3, 2, 1 → LL



예제 2 · 이어서 Insert 4, 5 → RR (pivot: 3)



시간복잡도: 삽입 $O(\log n)$ + 회전 위치 탐색 $O(\log n)$ + 회전 $O(1)$ → **총** $O(\log n)$.

4. Delete — 연쇄 불균형이 가능하다

① **표준 BST Delete**: leaf / 자식 1개 / 자식 2개(successor) 경우 처리.

② **불균형 확인**: 실제로 노드가 제거된 위치의 부모부터 루트까지 bf를 재계산한다. 자식이 둘인 노드를 삭제할 때는 successor를 복사한 뒤 successor가 원래 있던 자리에서 물리적으로 제거되므로, 재균형은 successor의 원래 부모 위치부터 시작한다.

③ **재균형**: 불균형 발견 시 회전. **회전으로 서브트리 높이가 줄면 불균형이 위로 전파될 수 있어 루트까지 계속 확인**한다.

> **Insert와 Delete의 차이 — 시험 단골**
>
> Insert는 회전 **최대 1회**로 끝나지만, Delete는 회전 후 서브트리 높이가 1 줄어들면서 상위 노드에 새 불균형을 만들 수 있다 — **연쇄 불균형(cascading imbalance)**, 회전이 여러 번 필요할 수 있다.

예제 · Delete(90)에서 연쇄 회전

① **삭제**: 90은 leaf라 그냥 제거.

```
          50(+1)
        /       \
     30(+1)      80(+1)
    /  \       /   \
  20    40    70     90 ← 삭제
 / \    \   /
10  25   45 60
/
5
```

② **1차 불균형 (노드 80)**: 오른쪽이 비어 bf(80) = +2 (LL) → 우회전. 서브트리 루트가 70으로, 높이가 2 → 1로 감소.

```
    70
   /  \
  60   80
```

③ **2차 불균형 (루트 50)**: 높이 감소가 위로 전파 → bf(50) = h(30) − h(70) = 3 − 1 = +2 (LL) → 루트에서 우회전.

```
        30
      /    \
    20      50
   /  \    /  \
  10   25 40    70
 /       \ / \
5        45 60  80
```

최종 루트 30의 양쪽 높이가 2로 같아져 bf = 0, 완전 균형 회복.

5. 연습: Insert 3, 2, 1, 4, 5, 6, 7 — 회전 횟수 세기

순서대로 삽입하며 매번 bf를 확인한다.

① **Insert 3, 2** — `3 → 2`. 불균형 없음.

② **Insert 1 → LL 불균형 (pivot 3)** — 우회전(1회째). 루트가 2로: `2 / 1,3`.

③ **Insert 4** — 3의 오른쪽에 부착. bf(3) = −1, 균형 유지. 회전 없음.

④ **Insert 5 → RR 불균형 (pivot 3)** — 5는 4의 오른쪽에 붙고 bf(3) = −2. 좌회전(2회째): `2 / 1, 4(3,5)`.

⑤ **Insert 6 → 루트에서 RR 불균형 (pivot 2)** — 6은 5의 오른쪽에 붙고, 오른쪽 서브트리 높이가 2가 되어 bf(2) = −2. 루트 좌회전(3회째): `4 / 2(1,3), 5(−,6)`.

⑥ **Insert 7 → RR 불균형 (pivot 5)** — 7은 6의 오른쪽에 붙고 bf(5) = −2. 좌회전(4회째): 최종 `4 / 2(1,3), 6(5,7)`, 모든 노드 균형.

회전 "횟수"뿐 아니라 **어느 노드가 pivot인지, LL/RR/LR/RL 중 무엇인지**를 함께 쓰는 문제가 많다. bf 부호로 유형을 먼저 판정하자: bf = +2면 왼쪽이 무거운 것(LL 또는 LR), −2면 오른쪽(RR 또는 RL).

6. Quiz

Q1. Insert 3, 2, 1, 4, 5, 6, 7에서 총 회전 횟수는?

힌트

섹션 5의 단계를 그대로 따라가며 |bf| = 2가 되는 순간을 센다.

정답

① Insert(1): LL → 우회전 1회 (pivot 3).

② Insert(5): RR → 좌회전 1회 (pivot 3).

③ Insert(6): RR → 좌회전 1회 (pivot = 루트 2).

④ Insert(7): RR → 좌회전 1회 (pivot 5).

정답: 총 4회 (LL 1회 + RR 3회).

Q2. 왜 AVL에서는 연산이 끝난 뒤 모든 노드의 |bf| ≤ 1이 보장되는가?

힌트

모든 Insert/Delete 직후에 하는 공통 작업.

정답

정답: Insert/Delete 과정에서는 일시적으로 bf가 ±2가 될 수 있다. 삽입·삭제 지점에서 루트까지 거슬러 올라가며 bf를 재계산하고, |bf| = 2인 노드를 발견하면 회전으로 복구한다. 이 재균형이 끝난 뒤에야 모든 노드에서 |bf| ≤ 1이 보장된다.

Q3. LL과 LR은 모두 왼쪽이 무거운데(bf = +2) 처리가 다른 이유는?

힌트

왼쪽 자식의 bf 부호를 본다. 안쪽으로 꺾인 형태는 한 번의 회전으로 안 펴진다.

정답

정답: LL은 왼쪽 자식의 bf가 0 이상이다(bf = +1이면 왼쪽이 무겁고, 삭제 후 재균형에서는 bf = 0인 경우도 포함한다). 그래서 우회전 1회로 해결되지만, LR은 왼쪽 자식의 bf가 음수라 먼저 왼쪽 자식을 좌회전해 LL 형태로 만든 뒤 부모를 우회전해야 한다.

Q4. Insert는 최대 1회 회전인데 Delete는 여러 회전이 필요할 수 있는 이유는?

힌트

회전 후 서브트리의 "높이"가 어떻게 변하는지 생각해 보자.

정답

정답: Delete 후 회전은 서브트리 높이를 1 줄여 상위 노드에 새 불균형을 전파할 수 있어 루트까지 반복 확인이 필요하다. Insert 후 회전은 서브트리 높이를 삽입 전으로 되돌려 전파가 없다.

Q5. AVL이 Red-Black Tree보다 높이는 낮은데, 실무에서 Red-Black이 더 많이 쓰이는 이유는?

힌트

균형 조건이 빡빡할수록 검색은 빠르지만 삽입·삭제 비용은 커진다.

정답

정답: 두 트리 모두 높이가 O(log n)이지만, Red-Black Tree는 균형 조건이 더 느슨해 삽입·삭제 때 높이 갱신과 회전이 대체로 적게 들고 구현·라이브러리에서 널리 쓰인다. 반대로 AVL은 높이가 더 엄격하게 제한되어 검색이 많은 경우에는 유리할 수 있지만, 삽입·삭제가 잦으면 재균형 비용이 커질 수 있다.

Q6. 회전 후에도 불균형이 남을 수 있는가?

힌트

Q4와 같은 원리 — 어느 연산에서 전파가 생기는가.

정답

정답: Insert 후 회전은 그 지점에서 완전히 해결되지만, Delete 후 회전은 부모 높이 변화로 상향 불균형이 전파될 수 있어 루트까지 계속 확인해야 한다.

7. 핵심 정리

- AVL Tree = BST + Height-Balanced (모든 노드 $|\text{bf}| \leq 1$)
- 높이 보장: $h \leq 1.44 \log_2 n$ → 검색/삽입/삭제 모두 $O(\log n)$
- 불균형 유형: LL(우회전), RR(좌회전), LR(좌+우), RL(우+좌)
- 회전 후 BST 성질(inorder 순서)은 항상 유지
- Insert: 최대 1회 회전 · Delete: 연쇄 회전 가능
- 검색 많음 → AVL, 삽입·삭제 많음 → Red-Black